Reposición asistida de repuestos mediante aprendizaje automático y coordinación multiagente en la nube

Equipo del proyecto RODA · Ingeniería de Sistemas

Informe técnico · 23 de octubre de 2026 · America/Lima

Resumen

La reposición de repuestos exige conciliar ventas, existencias, plazos de entrega, costos y restricciones de compra. Este trabajo diseñó y evaluó un prototipo académico de tienda web que integra pronóstico de demanda y coordinación multiagente para preparar decisiones administrativas verificables. Se implementó una aplicación React y FastAPI con persistencia PostgreSQL, cinco modelos de regresión comparados con referencias simples y ocho roles especializados que consultan servicios Python. La evaluación combinó una serie pública de ventas mensuales de repuestos, casos aislados de agentes y controles de permisos, errores y trazabilidad. En la comparación histórica mensual disponible, el promedio de doce meses obtuvo MAE de 0,541 unidades por serie y mes; V1 y V2 obtuvieron 0,575 y 0,571. En una demostración aislada, los cinco casos de control resultaron satisfactorios y la propuesta de compra permaneció pendiente de aprobación humana. Los resultados prueban la integración y sus controles bajo las condiciones ensayadas, pero no acreditan precisión de demanda para los productos actuales de RODA ni una ventaja estadística del equipo multiagente frente a reglas. La arquitectura conserva evidencia para evaluaciones posteriores con ventas observadas.

Palabras clave: pronóstico de demanda; repuestos automotrices; sistemas multiagente; inventario; evaluación temporal

1 Introducción

1.1 Problema y contexto

En una tienda de repuestos, la decisión de reponer no depende únicamente de que un producto tenga pocas unidades. Es necesario distinguir stock físico de reservas, considerar compras pendientes, comprobar si una oferta sigue vigente y determinar cuánto permite el presupuesto. Una decisión apresurada puede duplicar un pedido a proveedor o dejar un faltante sin advertir. El presente estudio aborda esa tarea administrativa en un prototipo académico peruano denominado RODA, sin atribuir sus registros de ejemplo a una empresa real.

El diagnóstico provino de revisar los flujos implementados del propio prototipo: catálogo, pedidos, kardex, proveedores y panel de administración. Se identificó la necesidad de convertir registros dispersos en una propuesta examinable por una persona responsable. La pregunta técnica fue si un pronóstico, cuando su evidencia fuese válida, y un equipo de roles con herramientas acotadas podían preparar una recomendación reproducible sin asumir la autoridad para ejecutar una compra. No se realizaron encuestas ni se midieron procesos de una empresa externa.

La salida esperada especifica producto, cantidad, proveedor, importe, plazo, advertencias y fundamento. Registra herramientas consultadas y observaciones resueltas. Si falta un pronóstico válido, aplica la política de inventario; si cambian stock u oferta antes de aprobar, exige un análisis nuevo. Estos criterios hacen evaluable el prototipo.

1.2 Antecedentes y fundamento

El archivo Monash reúne series de distintas frecuencias y publica referencias de pronóstico para facilitar comparaciones entre métodos; su aporte metodológico es separar el comportamiento de una serie de la fama de un algoritmo (Godahewa et al., 2021). El conjunto Car Parts asociado contiene 2674 series mensuales intermitentes de ventas de repuestos entre enero de 1998 y marzo de 2002 (Godahewa et al., 2020). Es una fuente externa útil para estudiar la evaluación temporal, aunque sus identificadores anónimos no corresponden a los SKU de RODA ni permiten inferir precios, stock o demanda peruana actual.

En pronóstico, una referencia ingenua o promedio histórico sirve para comprobar si la complejidad adicional aporta valor; el error se calcula sobre observaciones posteriores al entrenamiento (Hyndman y Athanasopoulos, 2021). Además, imputar o transformar datos antes de separar entrenamiento y prueba puede filtrar información futura y producir resultados optimistas (Scikit-learn developers, s. f.-a). Por ello, el trabajo conserva periodos distintos para entrenamiento, actualización y comparación, y no reemplaza valores de demanda desconocidos por ventas inventadas.

En sistemas multiagente, AutoGen presenta agentes conversacionales configurables que combinan herramientas, modelos y participación humana (Wu et al., 2023). Se tomó como antecedente conceptual de cooperación, no como biblioteca instalada. RODA usa una implementación propia: un orquestador selecciona especialistas, cada especialista interpreta datos de un servicio Python y el auditor revisa el expediente. Se trata de un flujo secuencial y

jerárquico con decisiones acotadas, no de ocho modelos distintos ni de una afirmación general de superioridad del paradigma multiagente.

Objetivos verificables

1. Diseñar e implementar una tienda web académica que registre ventas, inventario y propuestas administrativas trazables, con separación entre interfaz, servicios y persistencia.

2. Comparar cinco modelos predictivos con referencias históricas mediante una división temporal y establecer cuándo un resultado puede o no respaldar una reposición.

3. Evaluar una coordinación multiagente de ocho roles mediante casos normales, fallos controlados, permisos y comparación exploratoria con reglas y un solo agente.

2 Materiales y métodos

2.1 Diseño y alcance del estudio

Se realizó un desarrollo tecnológico aplicado con evaluación exploratoria. La unidad de análisis predictivo fue una serie de ventas por identificador y periodo; la unidad de análisis multiagente fue una ejecución administrativa con entradas, pasos, controles y salida. Para la comparación mensual se utilizaron datos públicos observados; para las pruebas de flujos de compra se usó una base temporal con productos y ofertas académicas identificadas. Ambos ámbitos permanecen separados. La evaluación no pretendió estimar el beneficio económico de una tienda en operación.

Se establecieron criterios previos: registrar la procedencia de datos, reservar periodos futuros, comparar MAE con una referencia simple, impedir que una propuesta exceda el presupuesto y mantener aprobación humana antes de crear órdenes de compra. Los casos de error incluyeron una oferta inválida, evidencia de stock cambiada, respuesta JSON incorrecta y fallo temporal del proveedor LLM. El éxito de una prueba se definió por controles observables, no por una explicación textual persuasiva.

2.2 Arquitectura y controles cloud

La Figura 1 muestra el recorrido entre interfaz, servicios, persistencia y componentes inteligentes. El modelo predictivo y el LLM tienen contratos distintos. El primero calcula cantidades; el segundo interpreta consultas y devuelve decisiones estructuradas que el servidor valida.

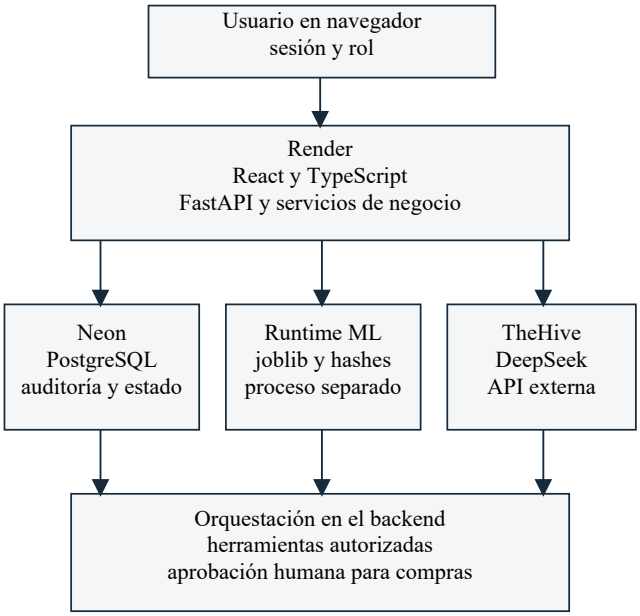


Figura 1. Arquitectura del prototipo desplegado. Fuente: elaboración propia a partir del código y render.yaml.

La Figura 1 representa cinco límites: navegador, API, servicios de negocio, persistencia y proveedores externos. El frontend React con TypeScript presenta tienda y área administrativa; FastAPI autentica las solicitudes y deriva el trabajo a módulos Python. PostgreSQL en Neon conserva productos, pedidos, movimientos, ofertas, sesiones y ejecuciones de agentes. El servicio web y la compilación del frontend están definidos en render.yaml para Render. El entorno gratuito es apropiado para demostraciones, pero puede suspenderse tras inactividad y su sistema de archivos no ofrece persistencia permanente (Render, s. f.).

Los servicios Python calculan cantidades y controles autoritativos. El proveedor de lenguaje devuelve decisiones estructuradas en JSON, pero no ejecuta SQL arbitrario ni crea una orden por sí mismo. El orquestador selecciona un equipo según tipo de tarea; cada especialista recibe mensajes breves y la salida de herramientas permitidas para su rol. El auditor recibe un expediente que reúne plan, alternativas, calidad, finanzas, advertencias y correcciones. agent_runs conserva estado, llamadas, tokens y checkpoint; agent_steps conserva cada consulta, mensaje y revisión.

Los agentes de reposición tienen una dependencia causal: Calidad comprueba inconsistencias, Inventario calcula necesidad, Compras compara alternativas, Finanzas revisa límite y margen, y Auditor verifica el expediente. Demanda ML se incorpora únicamente si existe pronóstico operativo vigente; Pedidos y envíos se incorpora si hay incidencias. Otros tipos de tarea, como catálogo o informe financiero, usan equipos más pequeños. El administrador puede iniciar, cancelar o retomar un

análisis fallido; una revisión diaria opcional solo encola un análisis, sin emitir compras.

El acceso a las rutas de agentes exige sesión y rol administrador. La cookie es HttpOnly y SameSite=Lax, con atributo Secure bajo HTTPS. Las escrituras rechazan orígenes ajenos y las variables de conexión o proveedor se leen del entorno. La propuesta validada queda en estado de revisión; aprobarla requiere un acto explícito del administrador y una transacción que vuelve a comprobar vigencia de ofertas e inventario. La interfaz no debe confundir este paso con pago, recepción o entrega automática.

La persistencia externa se apoya en la separación de cómputo y almacenamiento de Neon (Neon, s. f.). La ejecución de evaluación registró Python 3.12.10, scikit-learn 1.7.2, numpy 2.2.6, pandas 2.2.3 y joblib 1.5.1. Los contratos de datos y las dependencias fijadas permiten repetir los cálculos con esas versiones.

2.3 Datos y modelo predictivo

La fuente externa principal fue Car Parts, publicada en Zenodo por Godahewa y colaboradores. Registra 2674 series mensuales anónimas y contiene ausencias; el archivo preparado documenta 136374 filas y 6122 valores faltantes. El historial inicial, los datos incorporados después y las ventas posteriores se guardaron por separado, con sumas de verificación. Los signos de ausencia permanecieron como datos desconocidos. Se excluyeron objetivos no observados de la comparación y se imputaron entradas solo dentro del entrenamiento. No se asignaron series externas a productos del catálogo ni se derivaron precios o existencias inexistentes.

La variable objetivo fue la cantidad vendida en el siguiente mes para cada serie. Las entradas procedieron únicamente de ventas anteriores al corte: rezagos de uno, dos, tres, seis y doce meses; promedio, dispersión, venta reciente, proporción de ceros, número de faltantes y posición estacional del mes. El modelo no recibió el valor posterior que debía predecir. La prueba final empleó el mismo conjunto de 15054 observaciones, correspondientes a 2509 series y seis meses entre octubre de 2001 y marzo de 2002, para comparar versiones y promedio histórico.

Se evaluaron cinco familias: Ridge como relación lineal regularizada, Poisson para conteos, Random Forest y Extra Trees para interacciones no lineales, e Hist. Gradient Boosting como ensamble secuencial. Los hiperparámetros se fijaron o eligieron usando historial anterior a la prueba. V1 se entrenó con el primer tramo; V2 añadió meses conocidos y volvió a ajustar el modelo. La referencia fue el promedio histórico de los últimos doce meses observables. El MAE en unidades por serie y mes fue la métrica primaria; RMSE, WAPE, sesgo y R^2 ayudaron a interpretar errores sin redefinir el ganador tras mirar la prueba.

La web tiene además un contrato independiente para un pronóstico de siete días por SKU de su catálogo académico. Allí se compararon cinco modelos y referencias con cortes temporales, y el artefacto desplegado identifica Ridge como candidato académico. Esa evaluación no debe mezclarse numéricamente con Car Parts mensual: cambian frecuencia, población, objetivo y unidad. El servicio de inventario solo aplica un pronóstico operativo de la fecha anterior, con hash de modelo coincidente y predicción sin observación; de otro modo usa mínimos, máximos y compras pendientes. La demostración de ML repite un pronóstico guardado en una base temporal y no representa un entrenamiento nuevo.

V1 utilizó enero de 1998 a diciembre de 2000 y validó julio a diciembre de 2000. V2 amplió el historial hasta septiembre de 2001 y validó abril a septiembre de 2001. La prueba reservó octubre de 2001 a marzo de 2002. Cada origen mensual requiere una ventana de doce meses con al menos ocho cantidades conocidas; los objetivos desconocidos no se imputan.

La Tabla 1 define la unidad de observación, los predictores y el tratamiento de valores desconocidos. La Tabla 2 detalla la configuración que se mantuvo fija durante la comparación.

Tabla 1. Variables del experimento mensual Car Parts

Variable	Definición y tratamiento
sku y periodo	Clave de serie anónima y mes AAAA-MM; identifican filas y cortes. El SKU no entra como predictor numérico.
unidades	Conteo mensual observado. Cero significa ausencia de ventas; vacío significa desconocido. Objetivo: unidades del mes siguiente.
lag1, lag2, lag3, lag6, lag12	Cantidades observadas a 1, 2, 3, 6 y 12 meses antes del origen. Nulos de entrada se imputan con la media dentro del pipeline.
mean, std, recent	Media y desviación de la ventana de 12 meses; media de los últimos 3. Unidad: unidades por serie y mes.
zero_share, missing	Proporción de ceros entre datos observados y número de faltantes en la ventana. Sin confundir desconocidos con cero.
month_sin y month_cos	Codificación cíclica del mes previsto. Valores adimensionales calculados con el calendario.

Tabla 2. Configuración común de los cinco candidatos

Algoritmo	Configuración fijada
Ridge	Media, indicador de faltantes, StandardScaler; alpha=10.

Algoritmo	Configuración fijada
Poisson	Media, indicador, escalado; alpha=0.1; max_iter=500.
Random Forest	60 árboles, profundidad 10, mínimo 5 por hoja; n_jobs=2; semilla 42.
Extra Trees	60 árboles, profundidad 10, mínimo 5 por hoja; n_jobs=2; semilla 42.
Hist Gradient Boosting	150 iteraciones, 31 hojas, regularización L2=1; semilla 42; imputación con media.

Los doce predictores y los hiperparámetros corresponden al laboratorio mensual, no al motor diario de referencia ni al conjunto extendido experimental. Todos los candidatos incluyen imputación con media e indicadores de faltantes. La selección usa el menor MAE de validación; el ganador se reajusta con los objetivos conocidos hasta el corte. Las estimaciones negativas se recortan a cero. El artefacto conserva variables, frecuencia, fecha, series admitidas y hash SHA-256.

MAE, RMSE y R^2 describen errores de regresión con propiedades distintas; la métrica principal permanece fijada antes de la prueba (Scikit-learn developers, s. f.-b). No se cambia el criterio después de observar qué alternativa gana.

2.4 Agentes y supervisión humana

El sistema define ocho roles no redundantes: Orquestador, Calidad de datos, Demanda ML, Inventario, Compras y proveedores, Finanzas, Pedidos y envíos, y Auditor. Cada rol dispone de una lista blanca de herramientas de lectura, una evidencia de entrada y una salida tipada que expresa continuar, revisar o bloquear. Todos pueden compartir el mismo modelo de lenguaje configurado; la especialización reside en su responsabilidad, contexto y herramienta autorizada. Los cálculos de stock, costo y límites permanecen en los servicios Python.

Orquestador consulta el contexto y fija estrategia de costo o rapidez. Calidad consulta errores y origen; Demanda interpreta vigencia de pronósticos sin extrapolar. Inventario produce candidatos y cantidades netas de reservas y compras en tránsito. Compras inspecciona ofertas activas con costo, lote mínimo, plazo y procedencia; si elige otra oferta, el servidor valida que el identificador figure entre las alternativas. Finanzas compara el plan con el presupuesto y detecta costos mayores o iguales al precio de venta. Pedidos identifica atrasos e incidencias sin cambiar estados. Auditor contrasta restricciones y observaciones antes de presentar la propuesta.

Una oferta etiquetada referencia_catalogo es un costo académico derivado del catálogo, no una cotización recibida. Si existe una oferta manual viable, la referencia no sustituye esa oferta. El plan de compras prioriza productos secuencialmente; por tanto, no asegura la combinación global óptima. Cuando el presupuesto solo cubre una parte, las unidades restantes quedan visibles como advertencia. El flujo puede solicitar una corrección limitada a Compras y una nueva lectura de Finanzas; una inconsistencia real o una alternativa dominada sin justificación bloquea el cierre.

El control técnico limita cada análisis a treinta llamadas LLM, seis minutos y veinte productos; cada intervención dispone de hasta tres turnos. Las decisiones JSON pasan por validación de esquema. Una selección inválida se devuelve para corrección acotada, mientras que un cambio real de inventario detiene el análisis. Se registran llamadas, errores y pasos sin atribuir a los textos del proveedor autoridad sobre la base de datos. La propuesta de reposición únicamente puede transformarse en órdenes mediante aprobación humana, con validación transaccional de datos vigentes e idempotencia.

Orquestador

Objetivo: Elegir equipo y estrategia, coordinar revisiones. Entrada: Tipo de tarea, presupuesto, candidatos, agotados, pronóstico operativo e incidencias. Herramienta autorizada: contexto. Salida: Delegación con equipo, estrategia, justificación y mensajes. Límite: Equipo obligatorio fijado por servidor; opcionales solo si hay evidencia; hasta tres turnos por intervención. Aceptación: Equipo contiene roles obligatorios y no inventa pronósticos ni incidentes.

Calidad de datos

Objetivo: Detectar inconsistencias antes de usar la evidencia. Entrada: Controles operativos, observaciones y, para catálogo, hallazgos resumidos. Herramienta autorizada: calidad. Salida: Conclusión de continuar, revisar o bloquear con errores observados. Límite: No modifica registros; resultados de herramienta calculados en Python. Aceptación: Errores bloqueantes identificados y transmitidos al auditor; no afirmar datos verificados fuera de los checks.

Demanda ML

Objetivo: Interpretar disponibilidad y vigencia del pronóstico operativo. Entrada: Contrato del modelo, fecha de corte, hash y corrida operativa. Herramienta autorizada: demanda. Salida: Conclusión sobre uso o no uso del pronóstico. Límite: Opcional; se convoca solo con pronóstico operativo; no entrena ni extrapola fuera del horizonte. Aceptación: Pronóstico del día anterior, hash vigente y producto sin observación; de otro modo se usa política de inventario.

Inventario

Objetivo: Explicar necesidades de reposición considerando stock, reservas y compras pendientes. Entrada: Disponibilidad, mínimos/máximos, ventas recientes, compras en tránsito y pronóstico válido si existe. Herramienta autorizada: inventario, demanda. Salida: Candidatos, cantidad necesaria y fundamento política/ML. Límite: Máximo 20 productos por análisis; no modifica stock ni presume recepción de compras abiertas. Aceptación: Necesidad calculada por servicio Python y diferencia entre stock físico, reservado y en tránsito visible.

Compras y proveedores

Objetivo: Comparar ofertas y preparar una propuesta de reposición. Entrada: Candidatos, presupuesto, ofertas vigentes con costo, lote, mínimo, plazo y fuente. Herramienta autorizada: ofertas, demanda. Salida: Plan con proveedor, cantidades, importe, cobertura faltante, alternativas y motivo de selección. Límite: Solo selecciona offer_id de opciones válidas; no inventa ofertas ni altera cantidades directamente; la corrección recalcula lotes en Python. Aceptación: Oferta activa/vigente, costo menor al precio de venta, presupuesto respetado tras ajuste; alternativas dominadas justificadas.

Finanzas

Objetivo: Revisar presupuesto y margen comercial de la propuesta. Entrada: Plan de compras, costo, precio de venta y presupuesto fijado por administrador. Herramienta autorizada: finanzas. Salida: Observación de sobrepresupuesto, SKU con pérdida o conformidad. Límite: Presupuesto es límite, no efectivo disponible; margen comercial no equivale a ganancia neta. Aceptación: Importe dentro del límite y sin costos mayores o iguales a precios; si excede, pide corrección a Compras.

Pedidos y envíos

Objetivo: Detectar pedidos abiertos con atraso, falta de seguimiento o incidencia. Entrada: Estados, fechas esperadas, pagos registrados y tracking. Herramienta autorizada: pedidos. Salida: Resumen de incidentes para revisión administrativa. Límite: Opcional en reposición; no confirma entregas, pagos ni envíos; no envía contacto del cliente al modelo. Aceptación: Incidencias derivadas de estados y fechas reales, sin alterar pedido.

Auditor

Objetivo: Verificar expediente y cierre antes de mostrar la propuesta al administrador. Entrada: Calidad, necesidad, ofertas, alternativas, finanzas, correcciones, advertencias y controles. Herramienta autorizada: auditoria. Salida: Dictamen pasar/bloquear con errores y compromisos pendientes. Límite: No aprueba compras; el servidor exige auditoría válida y aprobación posterior por admin. Aceptación: Sin error bloqueante, dentro de presupuesto, ofertas/stock vigentes y observaciones resueltas; si falla, bloquea.

2.5 Protocolo de evaluación

La evaluación predictiva separó cronológicamente entrenamiento, actualización y comparación posterior. Se calculó MAE sobre los mismos pares serie-mes para V1, V2 y referencia, sin seleccionar una versión con filas distintas. Para analizar la repetibilidad se conservaron notebooks, archivos de entrada, hashes, parámetros, modelos serializados y salidas de evaluación. Esta estructura permite ejecutar la carga de datos adicionales sin alterar el historial inicial ni prometer que la siguiente versión mejorará automáticamente.

En agentes se prepararon cinco casos académicos aislados: ajuste por presupuesto, contraste costo-plazo de proveedores, preguntas difíciles al asistente, reserva inconsistente y efecto de un pronóstico vigente sobre la cantidad. La ejecución usa SQLite temporal y guarda un resultado resumido en la aplicación. Tres controles adicionales verifican presupuesto, compra en tránsito y reserva sin pedido con reglas Python, sin llamadas LLM. Para contraste exploratorio se repitieron tres escenarios en modalidades reglas, un agente y equipo completo, registrando estado, llamadas, tokens y segundos.

Se añadieron pruebas de permisos y fallos: empleado sin aprobación de compras, cambio de stock entre propuesta y aprobación, oferta vencida, rollback ante fallo del segundo proveedor, JSON inválido, selección de oferta ajena y caída temporal de API. Una prueba confirma que una petición maliciosa al asistente no crea venta ni cambia stock. La aceptación exige el resultado previsto y ausencia de efectos no autorizados. El tiempo de ejecución se informa como latencia observada de una configuración y no como garantía de desempeño en otras redes o proveedores.

El estudio nuevo fijó dos repeticiones por escenario y modalidad, límite de 360 s por análisis y controles de presupuesto, vigencia y ausencia de compras automáticas. Para las ocho preguntas del asistente se fijó un límite de 30 s por caso y se verificó información de la base, respuesta del proveedor y ausencia de costos privados.

Las pruebas automáticas de permisos, integridad y recuperación utilizan SQLite temporal y respuestas

controladas; se separan de las llamadas reales del estudio del proveedor. El criterio funcional exige satisfacer todos los controles del caso. El tiempo se mide de inicio a término, incluyendo proveedor. Los percentiles de una muestra pequeña no se interpretan como garantía de servicio.

El 23 de octubre se repitieron 41 pruebas de agentes en 13.137 s y 11 del asistente en 1.555 s; todas finalizaron correctamente. Cubrieron rechazo por rol, aprobación atómica e idempotente, stock cambiado, JSON inválido, recuperación acotada y ausencia de costos privados en respuestas al cliente. Estos tiempos corresponden a pruebas locales con proveedor aislado, no a latencia del LLM real. Los archivos pruebas-agentes.txt y pruebas-asistente.txt conservan nombres, resultado y duración.

3 Resultados y discusión técnica

3.1 Evaluación temporal del aprendizaje automático

La Tabla 3 compara las mismas 15,054 observaciones posteriores de 2,509 series, entre 2001-10 y 2002-03. El avance temporal incorpora únicamente ventas ya observadas antes de cada mes. No constituye una predicción fija de seis meses realizada de una sola vez.

Tabla 3. Prueba temporal común y referencia histórica

Método	MAE	RMSE	R²
V1	0.5747	1.0340	0.1023
V2	0.5710	1.0331	0.1038
Promedio	0.5411	1.0460	0.0812

V1 eligió Hist. Gradient Boosting y V2 eligió Extra Trees usando sus validaciones respectivas. El menor MAE de prueba corresponde a Promedio. Un cálculo válido o un modelo ganador entre cinco candidatos no equivalen a superar una referencia simple. MAE se expresa en unidades por serie y mes; WAPE no es un porcentaje de exactitud.

El 79.53% de los objetivos de prueba son cero. Predecir siempre cero obtiene MAE 0.3867, inferior a los métodos de la Tabla 3, pero omite todas las ventas positivas. Se conserva como diagnóstico de dispersión, no como política de reposición. El promedio es la referencia operativa predefinida; que los modelos tampoco superen esta referencia exige mantenerlos como exploratorios.

La Figura 2 representa el MAE de las dos versiones y de la referencia sobre la misma prueba temporal.

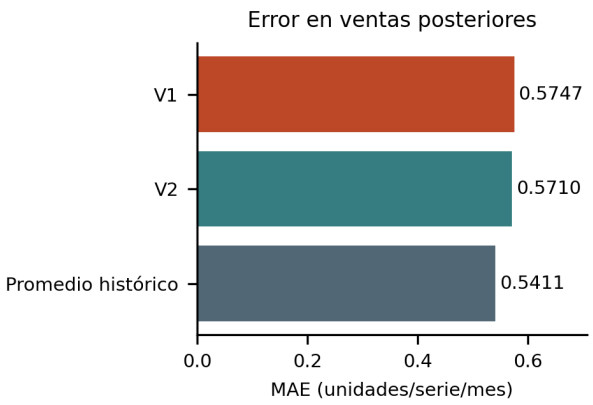


Figura 2. Error de predicción y comparación de versiones. Menor MAE es mejor. Fuente: evaluación temporal reproducible.

La ejecución completa del 23 de octubre duró 469.53 s localmente: 11 celdas sin error, diez ajustes para seleccionar cinco candidatos por versión y dos reajustes finales. La lectura del Excel verificó 24,066 filas contra el CSV original, incluidos los faltantes. MAE, RMSE y R² reprodujeron exactamente los resultados anteriores. Se comprobó la preservación SHA-256 de 17 archivos originales. El cuaderno ejecutado conserva estas salidas; no se atribuye su ejecución a Colab ni a Render.

3.2 Comparación de modalidades y coordinación

La Tabla 4 presenta cada ejecución del equipo. Debajo se resumen las mismas pruebas con reglas y con un solo agente.

Tabla 4. Casos del equipo multiagente y resultado observado

Caso y rep.	Estado	Tiempo s	Solicitudes
presupuesto 1	Cumple	58.58	8
presupuesto 2	Cumple	75.19	9
ofertas 1	Cumple	50.09	7
ofertas 2	Cumple	58.59	8
vencidas 1	Cumple	32.66	5
vencidas 2	Cumple	33.83	5

reglas: 6/6 cumplen; tiempo medio 0.06 s; 0 solicitudes al proveedor.

un_agente: 4/6 cumplen; tiempo medio 23.19 s; 12 solicitudes al proveedor.

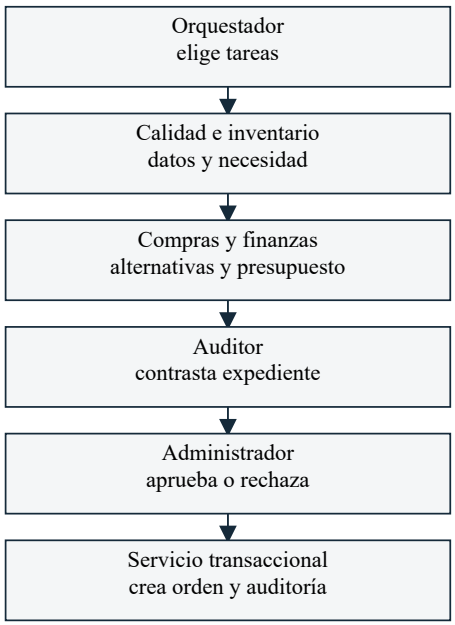
multiagente: 6/6 cumplen; tiempo medio 51.49 s; 42 solicitudes al proveedor.

Resultado esperado: presupuesto, propuesta dentro del límite; ofertas, selección válida; vencidas, bloqueo sin compras. Cada fila se identifica por caso y repetición en agents/entrega-2026-10-23.json, dentro de los anexos. Se conservan entrada, criterios, resultado, proveedor, fecha, versión y evidencia. Los tokens desconocidos de intentos fallidos no se contabilizan como gratuitos.

La comparación nueva contiene 18 ejecuciones. Los casos prueban presupuesto, alternativas de proveedor y vigencia de ofertas. Las tres modalidades

comparten servicios y controles Python. El denominador incluye fallos de formato y errores del proveedor, sin descartarlos. Los tiempos y tokens completos se incluyen en el archivo de evidencia.

La Figura 3 explica qué se coordina: una necesidad de inventario, alternativas de compra, restricciones y revisión. Los agentes utilizan el mismo proveedor LLM con contextos y herramientas diferentes; no son ocho modelos entrenados independientemente. Las decisiones observables y las herramientas se registran, sin publicar razonamiento interno del proveedor.



Demanda y pedidos participan si la tarea los requiere.

Figura 3. Coordinación para preparar una compra supervisada. Fuente: orquestador y servicios del prototipo.

En la demostración aislada de Render del 22 de octubre, los cinco casos de control se completaron. El caso de presupuesto propuso S/ 40.00 dentro de un límite de S/ 50.00 y conservó la aprobación humana. La exportación registra ocho llamadas del equipo. Esta prueba usa datos académicos temporales y no representa una compra comercial ni demuestra ahorro operativo real.

El siguiente fragmento adapta las llamadas del comparador a los servicios compartidos; la creación de una compra permanece en otra operación autenticada.

```
stock = data.candidates(5)
state['plan'] = data.purchase_plan(
    stock, 'costo', budget, True)
state['audit'] = data.audit_result(
    run, state)
```

3.3 Evaluación del asistente de la tienda

La Tabla 5 presenta los ocho casos del asistente, su cumplimiento y la latencia observada con el proveedor real.

Tabla 5. Consultas del asistente y controles observados

Caso	Resultado	Tiempos
precio	Cumple	4.875
incompatible	Cumple	3.438
ambiguo	Cumple	1.672
contexto	Cumple	2.75
seguimiento	Cumple	2.844
inexistente	Cumple	3.047
manipulacion	Cumple	3.562
tienda	Cumple	3.219

Se registraron 8 casos, de los cuales 8 cumplieron los criterios definidos. La evaluación conserva solicitudes, verificaciones y proveedor efectivo sin claves. El catálogo es un conjunto de prueba aislado; el servicio no inventa ventas ni modifica el stock para responder consultas.

3.4 Evidencia del despliegue

La Tabla 6 separa consultas y pruebas completadas en Render de los pasos que no finalizaron en la instancia gratuita.

Tabla 6. Recorrido comprobado en el sistema desplegado

Prueba	Estado y alcance
Arranque tras suspensión	23/10: Render mostró su página de arranque; después abrió el laboratorio con sesión administradora.
Persistencia del laboratorio	23/10: se restauraron 120,330 observaciones, V1 y el pronóstico guardado del 22/10.
Excel y predicción V1	22/10: importación de 24,066 filas; pronóstico guardado con 2,509 resultados válidos y CSV descargado.
Intento de entrenamiento V2	23/10 a las 11:34:51: se inició en Render. Terminó como Revisar: el servidor se reinició. No se obtuvo V2 en esta prueba cloud.
Evaluación posterior cloud	No ejecutada: depende de que termine V2. El ciclo completo se entrenó y evaluó localmente el 23/10, con notebook ejecutado adjunto.
Agentes y asistente	Exportación Render del 22/10: 5/5 casos aislados. El 23/10 se consultó la propuesta guardada: seis agentes, 23 unidades, S/ 1,617.21 de S/ 2,000, por aprobar y sin pronóstico operativo.
Asistente web 23/10	Consulta RP-010-01: respuesta TheHive/DeepSeek, S/ 160.00, 14 disponibles y compatible con Toyota Yaris 2018 según catálogo. Sin errores de consola capturados.

La disponibilidad depende del plan y de la API externa. Render Free puede suspender la instancia por inactividad y su disco es efímero (Render, s. f.). El

prototipo persiste el laboratorio y sus modelos en Neon y los restaura al iniciar. Esta recuperación no equivale a garantizar un planificador activo durante toda la jornada.

La prueba de V2 del 23 de octubre no llegó a una versión guardada en Render: el historial informó un reinicio del servidor. Por ello, no se ejecutó allí la comparación posterior. No se infiere la causa exacta del reinicio sin registros suficientes. La evidencia reproducible local cierra el ciclo completo; para una exposición cloud, el entrenamiento grande debe ejecutarse fuera de la instancia gratuita o en un entorno cuya capacidad se verifique antes.

3.5 Interpretación y limitaciones

La separación entre decisiones de lenguaje y cálculos autoritativos permitió inspeccionar por qué una propuesta pasó, se corrigió o se bloqueó. Sin embargo, la validez de una cadena de agentes no se deduce de que todos hayan escrito mensajes: importan las herramientas consultadas, los valores contrastados y el estado final. Los controles de presupuesto y auditoría habrían funcionado también como reglas Python; la aportación observable de la orquestación es organizar especialidades y dejar un expediente legible, a cambio de más llamadas y latencia.

La evidencia predictiva real es histórica y mensual, mientras que la reposición web necesita estimaciones por SKU y horizonte corto. Car Parts carece de nombres de piezas, precio, existencias y país verificable para cada serie. No es correcto transferir su MAE al catálogo de RODA ni afirmar que V2 aprendió de ventas actuales. El historial de la tienda es académico y la propia pantalla distingue pronósticos operativos de demostraciones. Una evaluación futura debería recopilar cierres observados, verificar calidad y ausencia de quiebres no registrados y repetir pruebas temporales antes de autorizar cambios de modelo.

El proveedor LLM puede fallar por latencia, formato o disponibilidad. Se acotaron llamadas, turnos y reintentos, y los fallos se registran sin aprobar una compra. El plan gratuito de Render puede suspender el servicio inactivo y no conserva archivos locales tras reinicios; PostgreSQL externo mantiene datos persistentes, pero una revisión programada depende de que el servicio esté activo o vuelva a iniciarse. Por ello, una demostración académica debe comprobar el estado del despliegue antes de ejecutarse y conservar exportaciones de resultados sin secretos.

La comparación nueva del 23 de octubre amplió a dos repeticiones por caso y modalidad: reglas 6/6, agente único 4/6 y equipo 6/6. El agente único agotó reintentos de formato en los dos casos de presupuesto; el equipo recuperó tres salidas JSON inválidas y completó los casos. La muestra comparte tres escenarios fijos, por lo que la diferencia observada es exploratoria y no demuestra superioridad estadística. El uso de llamadas, latencia y tokens debe valorarse junto con la revisión especializada.

Los fallos del agente único se registraron como ProviderUnavailable después de tres errores de decodificación JSON en cada caso. No se confirmó truncamiento ni una causa única del formato recibido; el estudio conserva el fallo observable sin atribuirlo a una causa no demostrada.

4 Conclusiones

Se implementó una aplicación web que integra catálogo, inventario, pedidos y propuestas administrativas con una capa de servicios Python y persistencia relacional. La separación permitió que herramientas de agentes reutilizaran cálculos de negocio y que una propuesta no modificara stock ni creara compras sin autorización. El alcance demostrado es el de un prototipo académico desplegado, no el de una tienda comercial evaluada en producción.

La comparación histórica mensual permitió entrenar y actualizar dos versiones sobre ventas observadas externas y compararlas con una referencia simple. En 15054 observaciones posteriores, V2 alcanzó MAE 0,571 y el promedio histórico 0,541; por ello, el experimento no justifica promover V2 como mejor método para ese conjunto. El resultado demuestra un criterio de selección prudente, sin validar precisión para SKU peruanos actuales ni para un horizonte semanal.

La coordinación de ocho roles produjo propuestas auditadas y mantuvo supervisión humana. En la comparación nueva, el equipo cumplió 6/6 ejecuciones, las reglas 6/6 y el agente único 4/6; este último presentó dos fallos de formato del proveedor. Los ocho casos del asistente cumplieron sus controles. Los resultados respaldan la integración bajo los escenarios ensayados, pero el mayor tiempo y número de llamadas impiden atribuir una ventaja general al equipo.

El aporte principal es una arquitectura evaluable con procedencia de datos, controles de acceso, límites de llamadas, detección de evidencia cambiante y trazabilidad de decisiones. Para avanzar se recomienda acumular ventas observadas, comparar pronósticos por SKU en varios cortes y repetir escenarios de agentes con más instancias y métricas de calidad además de tiempo y costo. Hasta entonces, las propuestas deben conservar la revisión del administrador.

Referencias

Godahewa, R., Bergmeir, C., Webb, G., Hyndman, R., y Montero-Manso, P. (2020). Car Parts Dataset (with Missing Values) (versión 2) [Conjunto de datos]. Zenodo. https://doi.org/10.5281/zenodo.4656022

Godahewa, R., Bergmeir, C., Webb, G. I., Hyndman, R. J., y Montero-Manso, P. (2021). Monash Time Series Forecasting Archive. arXiv. https://doi.org/10.48550/arXiv.2105.06643

Hyndman, R. J., y Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3.ª ed.). OTexts. https://otexts.com/fpp3/toolbox.html

Neon. (s. f.). Architecture overview. Recuperado el 23 de octubre de 2026 de https://neon.com/docs/introduction/architecture-overview

Render. (s. f.). Deploy for Free. Recuperado el 23 de octubre de 2026 de https://render.com/docs/free

Scikit-learn developers. (s. f.-a). Common pitfalls and recommended practices. Recuperado el 23 de octubre de 2026 de https://scikit-learn.org/stable/common_pitfalls.html

Scikit-learn developers. (s. f.-b). Metrics and scoring. Recuperado el 23 de octubre de 2026 de https://scikit-learn.org/stable/modules/model_evaluation.html

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., y Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv. https://doi.org/10.48550/arXiv.2308.08155

Anexo A Acceso y reproducción

Sistema: https://roda-repuestos.onrender.com
Repositorio: https://github.com/NEYEN111/REPUESTOS
Colab: https://colab.research.google.com/github/NEYEN111/REPUESTOS/blob/main/notebooks/RODA_ML_Datos_Reales.ipynb

El paquete adjunto incluye el cuaderno ejecutado, insumos CSV y Excel, modelos joblib de la evaluación, resultados JSON y archivos de pruebas. El archivo README indica dependencias, comandos y variables requeridas. Los identificadores de ejecución y hashes se conservan en los artefactos. Las credenciales de acceso se entregan por separado al docente; no se publican claves API en el informe.

Recorrido de exposición: mostrar la fuente y los cortes; abrir V1; importar el Excel; ejecutar o consultar V2; incorporar ventas posteriores; comparar con la referencia; abrir el expediente de agentes; explicar una corrección y el control del Auditor; consultar precio o compatibilidad en el asistente. Una función que no complete la prueba en Render se identifica en la Tabla 6 y se demuestra por separado con evidencia local.

Identificación académica: sustituir Equipo del proyecto RODA por los nombres de los integrantes y agregar universidad, curso y docente antes de entregar. Estos datos personales no se han inferido.